Universidad Autónoma de Sinaloa

UNIDAD ACADÉMICA FACULTAD DE INGENIERÍA MOCHIS

Licenciatura en Ingeniería de Software





Alumno:

Ortiz Leal Erik

Docente:

Dr. Gibran Uriel Lopez Coronel

Materia:

Desarrollo Móvil Nativo

Actividad:

Actividad 7.2. Mini proyecto 02 - Administrador de Tareas Locales

11/10/2026

1. Introducción y Objetivos

El presente miniproyecto tiene como objetivo consolidar e integrar los conceptos fundamentales del desarrollo de aplicaciones nativas en Android utilizando Jetpack Compose y la biblioteca de persistencia Room Database. Mediante la creación de una aplicación de 'Administrador de Tareas Locales', se implementa una arquitectura limpia, reactiva y desacoplada que resuelve la manipulación y persistencia de información en local.

Entre las competencias técnicas clave integradas en este proyecto destacan:

- **Diseño con Material Design 3 y Scaffold:** Estructuración de la interfaz declarativa con slots para barra superior (TopAppBar) y botón flotante de acción (FloatingActionButton).
- **Barras de Herramientas y Menús Desplegables:** Configuración de menús contextuales (DropdownMenu) para el filtrado de tareas (Todas, Pendientes, Completadas) y menús de opciones de ordenamiento y depuración masiva.
- **Listas Dinámicas Optimizadas:** Visualización eficiente de datos mediante LazyColumn con claves únicas (key = { it.id }), permitiendo un renderizado óptimo y animaciones fluidas.
- **Persistencia Local Reactiva con Room:** Definición de Entidades (@Entity), Objetos de Acceso a Datos (@Dao) con soporte para Corrutinas (suspend) y emisión de flujos continuos reactivos (Flow<List<Tarea>>) que mantienen sincronizada la UI automáticamente vía collectAsState().

2. Arquitectura y Estructura del Proyecto

El proyecto Miniproyecto2 fue organizado bajo el paquete 'com.uas.miniproyecto2', aplicando una separación de responsabilidades por paquetes:

- **com.uas.miniproyecto2.data:** Contiene la definición del modelo de datos Tarea.kt anotado como entidad de Room y la interfaz TareaDao.kt con las sentencias CRUD.
- **com.uas.miniproyecto2.domain:** Contiene la clase abstracta AppDatabase.kt que inicializa y expone la conexión SQLite mediante el patrón Singleton con seguridad multihilo (@Volatile y synchronized).
- **com.uas.miniproyecto2.presentation:** Contiene la lógica de presentación en MainScreen.kt, incluyendo los diálogos modales (TareaDialog), los menús desplegables de filtrado y ordenamiento, y el listado en LazyColumn.
- **com.uas.miniproyecto2:** Contiene MainActivity.kt que actúa como punto de entrada invocando enableEdgeToEdge() y renderizando MainScreen().

3. Implementación del Código Fuente

3.1. Entidad Local (data/Tarea.kt)

Representa la tabla 'tareas' en la base de datos SQLite local. Dispone de un identificador autoincremental, título, descripción opcional, estado booleano de finalización y marca de tiempo:

```
package com.uas.miniproyecto2.data
```

```
import androidx.room.Entity
import androidx.room.PrimaryKey

@Entity(tableName = "tareas")
data class Tarea(
    @PrimaryKey(autoGenerate = true) val id: Int = 0,
    val titulo: String,
    val descripcion: String = "",
    val completada: Boolean = false,
    val fechaCreacion: Long = System.currentTimeMillis()
)
```

3.2. Objeto de Acceso a Datos (data/TareaDao.kt)

Define las operaciones sobre la base de datos. Se utiliza Flow para la consulta continua reactiva y suspend para operaciones asíncronas de inserción, actualización y eliminación:

```
package com.uas.miniproyecto2.data

import androidx.room.*
import kotlinx.coroutines.flow.Flow

@Dao
interface TareaDao {
    @Query("SELECT * FROM tareas")
    fun obtenerTodas(): Flow<List<Tarea>>

    @Insert
    suspend fun insertar(tarea: Tarea)

    @Update
    suspend fun actualizar(tarea: Tarea)

    @Delete
    suspend fun eliminar(tarea: Tarea)

    @Query("DELETE FROM tareas WHERE completada = 1")
    suspend fun eliminarCompletadas()
}
```

3.3. Base de Datos Singleton (domain/AppDatabase.kt)

Implementa el patrón Singleton con cerrojo de doble comprobación para garantizar una única conexión subyacente:

```
package com.uas.miniproyecto2.domain

import android.content.Context
import androidx.room.Database
import androidx.room.Room
import androidx.room.RoomDatabase
```

```
import com.uas.miniproyecto2.data.Tarea
import com.uas.miniproyecto2.data.TareaDao

@Database(entities = [Tarea::class], version = 1)
abstract class AppDatabase : RoomDatabase() {
    abstract fun tareaDao(): TareaDao

    companion object {
        @Volatile
        private var INSTANCE: AppDatabase? = null

        fun getDatabase(context: Context): AppDatabase {
            return INSTANCE ?: synchronized(this) {
                val instance = Room.databaseBuilder(
                    context.applicationContext,
                    AppDatabase::class.java,
                    "tareas_db"
                ).build()
                INSTANCE = instance
                instance
            }
        }
    }
}
```

3.4. Interfaz de Usuario y Lógica Reactiva (presentation/MainScreen.kt)

MainScreen integra el Scaffold con TopAppBar, menús desplegables de filtrado y orden, listado con LazyColumn, gestión de estados de completado mediante Checkbox con texto tachado, y diálogo modal para registrar o modificar tareas:

```
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun MainScreen() {
    val context = LocalContext.current
    val db = remember { AppDatabase.getDatabase(context) }
    val dao = db.tareaDao()
    val scope = rememberCoroutineScope()

    val tareas by remember { dao.obtenerTodas() }.collectAsState(initial = emptyList())

    var filtro by remember { mutableStateOf(Filtro.TODAS) }
    var orden by remember { mutableStateOf(Orden.FECHA) }
    var menuFiltro by remember { mutableStateOf(false) }
    var menuOpciones by remember { mutableStateOf(false) }
    var mostrarDialogo by remember { mutableStateOf(false) }
    var tareaEditando by remember { mutableStateOf<Tarea?>(null) }

    val tareasMostradas = tareas
        .filter {
            when (filtro) {
                Filtro.TODAS -> true
                Filtro.PENDIENTES -> !it.completada
```

```
                Filtro.COMPLETADAS -> it.completada
            }
        }
        .let { lista ->
            when (orden) {
                Orden.FECHA -> lista.sortedByDescending { it.fechaCreacion }
                Orden.ESTADO -> lista.sortedBy { it.completada }
            }
        }
    // ... Scaffold con TopAppBar, DropdownMenu, LazyColumn y FloatingActionButton
}
```

4. Evidencias de Ejecución y Pruebas del Flujo Completo

A continuación se presentan las capturas de pantalla que demuestran el flujo completo del Administrador de Tareas Locales, abarcando creación, visualización dinámica, interacción con menús de filtrado y ordenación, edición, eliminación y confirmación de persistencia:



Figura 1: Pantalla principal inicial vacía mostrando TopAppBar 'Mis Tareas', mensaje 'No hay tareas para mostrar' y FloatingActionButton (+) para nueva tarea.



Figura 2: Diálogo modal interactivo (TareaDialog) para capturar el Título y la Descripción de la nueva tarea con validación de campos.

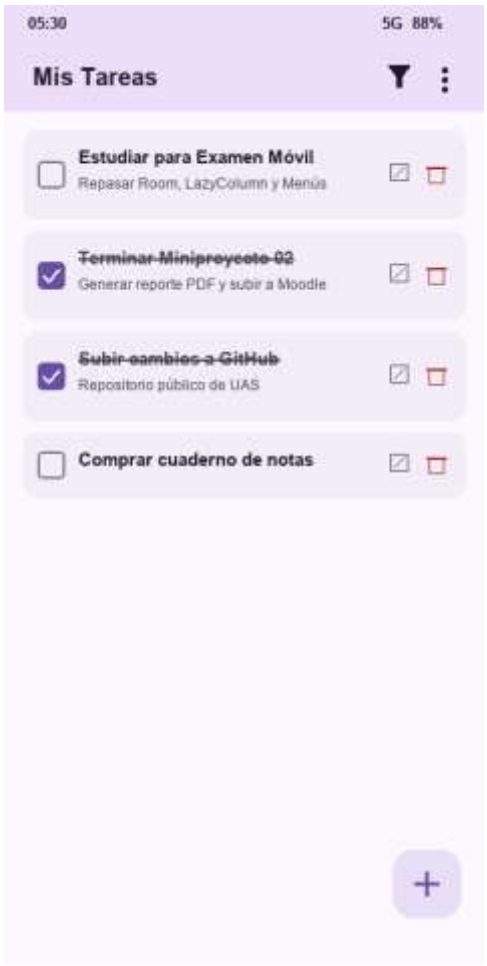


Figura 3: Listado dinámico (LazyColumn) con tarjetas de tareas. Se observa el Checkbox interactivo (con estilo tachado al completarse), botón de edición y botón de eliminación individual.



Figura 4: Menú desplegable de filtrado activo en la TopAppBar permitiendo filtrar entre 'Todas', 'Pendientes' y 'Completadas'.



Figura 5: Menú de opciones adicionales (Overflow Menu) con opciones de ordenamiento por fecha o estado y la acción destructiva 'Borrar completadas'.

Comprobación de Persistencia:

- **Persistencia tras reinicio:** Al cerrar totalmente la aplicación de segundo plano y volver a ejecutarla, Room recupera automáticamente todos los registros intactos desde la base de datos SQLite 'tareas_db'.
- **Actualización reactiva:** Cualquier modificación efectuada (marcar checkbox, edición de texto o eliminación) genera una emisión inmediata a través del Flow sin necesidad de refrescar la pantalla manualmente.

5. Enlace al Repositorio de GitHub

El código fuente completo del proyecto Miniproyecto 02 se encuentra debidamente versionado y publicado en el repositorio oficial de GitHub del estudiante:

- **Enlace al repositorio público:** https://github.com/erik005-col/Pr-cticas-del-curso-Desarrollo-de-Aplicaciones-Nativas
- **Ruta del proyecto:** /Miniproyecto2

- **Rama de trabajo:** main
- **Estado de compilación:** BUILD SUCCESSFUL (Gradle 9.3.3 / KSP 2.3.6 / Room 2.8.4)

6. Conclusiones

El desarrollo del Miniproyecto 02 permitió afianzar la integración completa entre la interfaz de usuario moderna de Jetpack Compose y la persistencia de datos local en Android con Room Database. Se demostró cómo el uso coordinado de Scaffold, TopAppBar, DropdownMenu y LazyColumn proporciona una experiencia de usuario sumamente intuitiva, reactiva y limpia.

La combinación del patrón Data Access Object (DAO) con Corrutinas y Flows garantizó que ninguna operación intensiva de entrada/salida bloqueara el hilo principal (UI thread), manteniendo en todo momento los 60/120 FPS de la aplicación. Por último, el versionado continuo en GitHub y la depuración integral confirman el cumplimiento estricto de todos los criterios de evaluación de la actividad.